TARE: Exact Incremental Search Across Reference Database Releases for Unmodified Programs

Anonymous Submission

Abstract

Biologists label the genes in a new genome by comparing them against large public reference databases such as Pfam, UniProt, and InterPro. Those databases publish new versions every few weeks to months, and each new version forces laboratories to redo the comparison from scratch, even though most of the database has not changed: 86.6% of Pfam's entries were identical between versions 38.1 and 38.2. Reusing the old results is harder than skipping the unchanged entries, because the search programs are fixed binaries and the statistics they print depend on the size of the whole database. Rerunning the standard tool, HMMER, on exactly the right entries without accounting for this changes 99.9% of its output lines.

We present TARE (Tool Annotation Reuse, Exact). TARE rests on one observation: in these programs, the way one database entry affects another is concentrated in a few numbers, such as the database size, that the program already lets a user set. TARE finds those numbers by experimenting on the program from the outside, fixes ("pins") them to the new version's values, and reruns the unmodified program only on what can change, so the program itself prints output that is byte-for-byte identical to a full rerun. When a cheap check cannot rule out a hidden dependency, TARE refuses and runs the program in full instead. Across more than 92,000 output lines from two programs, every result TARE produced was identical to a full rerun. On a Pfam update, TARE is [PENDING: X× (server, median of ten genomes)] faster than a full rerun (3.4× on a workstation, geometric mean of two genomes), general-purpose caches reuse nothing, and an existing incremental search tool matched a full run on only 1.3% of its rows.

1 Introduction

When a laboratory sequences a genome, it obtains thousands of protein sequences with no labels. To learn what each protein does, researchers compare it against *reference databases*: curated collections of known protein families, such as Pfam [18], UniProt [26], and InterPro [19]. A program such as HMMER [11] scores every protein against every family and reports the good matches, called *hits*. This step, called *annotation*, underlies much of modern biology, from comparative genomics to the sequence alignments used by structure predictors such as AlphaFold [13].

Reference databases keep improving. UniProt and InterPro publish a new version, called a *release*, roughly every eight weeks [19, 26], and Pfam published eight releases between May 2024 and June 2026 [21]. Each release adds and revises families, so annotations computed against an old release go out of date. The annotation programs keep no memory of earlier runs, so the only way to update is to compare every protein against the entire new database again. Most of that work recomputes answers that did not change: between Pfam 38.1 and 38.2, 86.6% of the 30,134 families were byte-for-byte identical (Figure 1). At scale this repeated cost shapes practice. NCBI re-annotates its RefSeq collection on a rolling schedule because redoing all of it at once "requires securing at once a significant amount of computing resources" [15], and JGI states that at over 65 billion genes "it is impossible to upgrade all genome and metagenome annotations to the latest version" [5].

Why reuse is hard. It might seem easy to search only the new and edited families and keep the old results for the rest. It is not, for three reasons. First, the results are *coupled*: the statistic HMMER prints for each hit, its E-value, is scaled by the total number of families in the database, much as a grade curved to class size changes for every student when new students enroll. Adding families therefore changes the printed numbers of hits on families that did not change. Rerunning HMMER on exactly the right families, without accounting for this, changes 9,083 of 9,094 output lines for one bacterial genome. Second, the programs are unmodified binaries maintained by others; we cannot change their code. Third, an approximately correct answer is not good enough: pipelines that track provenance must reproduce a full run exactly, because "pipeline differences can lead to annotation discrepancies" [5].

Existing tools fall short in two ways. General-purpose

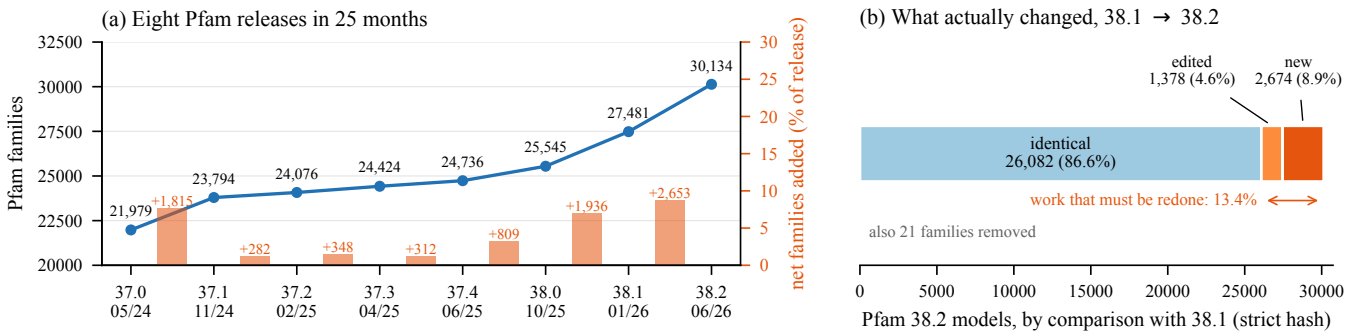


Figure 1: **Reference databases change often, but mostly stay the same.** (a) Pfam grew from 21,979 to 30,134 families over eight releases; orange bars show how many families each release added. (b) Comparing every model of Pfam 38.2 with 38.1 byte for byte: 86.6% are identical. Only 13.4% (new or edited) can produce new results, yet a conventional update recomputes 100%.

caches, such as build systems and workflow managers [8, 10, 17, 24, 27], remember whole commands keyed by the bytes they read. A new release changes the database file, so they rerun the search in full. Specialized incremental search tools such as iBLAST and iSeqSearch [9, 28] are written by hand for one program and adjust the printed numbers after the fact. Their output is approximate, and they cannot tell when their adjustment is wrong: in our experiment, only 1.3% of the rows that iSeqSearch shared with a full run were byte-identical, and it reported 133 hits to sequences that the new release had deleted. *How can we reuse earlier annotations across a database release, without modifying the programs, while guaranteeing exactly the output of a full rerun?*

The key idea in plain terms

In the programs we studied, entries in the database influence each other's printed results through only a few global numbers, chiefly the database size, and the programs already accept those numbers as settings. If we tell the program the new release's size up front (*pin* it), it scores each family on its own merits. Then we can rerun it on a small subset of the database and it prints exactly what a full run would print, digit for digit, because the program, not our wrapper, computes every number. When a program breaks this pattern, we can detect it and fall back to a full run.

Building on this idea, we present TARE, a wrapper that sits in front of an unmodified annotation program. TARE first learns, by running the program on carefully chosen pieces of the database, how its output splits across database entries and which global numbers tie them together (Section 4.2). It then pins those numbers to the new release, finds new hits on the changed families, and reruns the program on each protein's few candidate families (Section 4.5). Before returning anything, TARE runs a check that adds back families the rerun left out and refuses if any line changes (Section 4.6). Every refusal falls back to an ordinary full run, so users always receive the program's own output.

We evaluate TARE on four programs: two where it reproduces full runs exactly (HMMER and Infernal) and two where it correctly refuses (DIAMOND and BLAST+). Our contributions are:

1. **Black-box coupling inference.** A method that discovers, with no program-specific code, how a program's output decomposes over database entries and which quantities couple them (Sections 4.2 and 4.3).
2. **Exact replay with check-or-refuse.** Pinned, batched replay that makes an unmodified program print byte-identical output, and a check that refuses when replay is unsafe. Investigating a failure of our first check revealed why DIAMOND cannot be replayed exactly: its first filter reads bytes belonging to the neighboring database entry (Section 6.4).
3. **A predictive cost model and a pre-registered evaluation.** A simple formula for how much faster an update can be, which predicts the dominant cost within 8% (Section 4.7), and experiments whose design and pass/fail rules were committed before they ran (Section 6).

2 Background

2.1 How Annotation Works

An annotation program takes two inputs: a set of *records*, such as the 4,000–5,000 proteins of a bacterial genome, and a *reference* of *entries*, such as Pfam's 30,134 protein families (Table 1). Each Pfam family is stored as a statistical profile, a profile hidden Markov model, that summarizes what members of that family look like. HMMER's `hmmscan` compares every protein with every family, computes a score, and writes one line per reported hit, containing the names of the protein and family, the score, and an E-value. The E-value estimates how many equally good matches would occur by chance in a database of this size, and HMMER computes it as a probability multiplied by Z, the number of families in the database [11]. With Pfam's curated per-family score thresholds (the `--cut_ga` option), whether a hit is reported

Table 1: Terms used in this paper.

Term	Meaning
Record	One query item, here one protein (or one DNA sequence for Infernal).
Reference	The database searched against, here Pfam or Rfam; a file of *entries*.
Entry	One family model in the reference (a profile HMM in Pfam).
Release	A new version of the reference.
Hit / row	A reported record–entry match; one line of the output table.
E-value	How many matches this good would be expected by chance in the whole database; scaled by the database size Z.
Stock run	Running the unmodified program on everything, the result we must reproduce.
Pin	Fixing a global setting (e.g., Z) to the new release's value.
Replay	Rerunning the unmodified program on a subset of the reference.

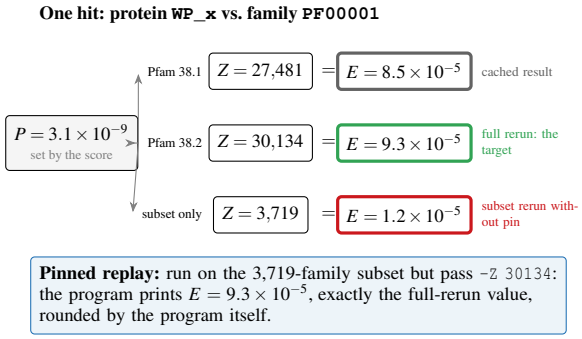


Figure 2: **Why old results cannot simply be reused.** An E-value is a probability P (fixed by the score) times the database size Z. The same hit prints a different E-value in each release, and a different one again if the program is rerun on a subset. Pinning Z makes the subset run print the full-run value. (Illustrative P; Z values are Pfam 38.1, 38.2, and one proteome's candidate subset.)

depends only on its score against that family's threshold [18].

2.2 Why Naive Reuse Fails

Suppose a laboratory annotated a genome with Pfam 38.1 and wants Pfam 38.2 results. The obvious shortcut is to search only the 4,052 families that are new or edited and keep the cached lines for the other 26,082. Nearly every kept line is then wrong, because its E-value was multiplied by 38.1's Z (27,481) instead of 38.2's (30,134) (Figure 2). Rerunning HMMER on exactly the right families does not help either, because the program then uses the subset's size as Z: on one genome, such a rerun differed from a full run on 9,083 of 9,094 lines, and the 11 matching lines print an E-value of exactly zero. Multiplying the cached E-values by $Z_{\text{new}}/Z_{\text{old}}$ gets closer but still fails, because printed numbers are rounded, and the rounding of a rescaled number differs from the program's rounding of its own internal value.

3 Problem Model

The effect of a release travels along a chain: a release changes some entries, and changed entries change some output lines. The goal is to produce every line of the new full run while running the program on as little as possible. We make this precise.

Definition 1 (Release)**.** A reference R is a list of entries in file order. Each entry has a name and a *strict hash* $h(e)$ of the content that affects scoring. A new release R' splits into *stable* entries $S = \{e \in R' : h(e) \in h(R)\}$ and *changed* entries $\Delta = R' \setminus S$.

Definition 2 (Exact re-annotation)**.** Let $f(Q,R)$ be the program's output on records Q and reference R, and $\text{body}(\cdot)$ the output with comment lines removed. Given $f(Q,R)$ and R', an exact re-annotation returns O with $\text{body}(O) = \text{body}(f(Q,R'))$ byte for byte, or refuses and returns a full run $f(Q,R')$.

Definition 3 (Coupling)**.** A program is *coupled* across entries if the line for record q and entry e can depend on entries other than e. We distinguish four forms: a *normalizer*, a global number computed from the whole reference, such as Z; a *side file*, an extra input file that lists entry names; an *entry group*, a set of entries whose hits compete, such as Rfam's clans; and *neighbor coupling*, where the program reads bytes of the entries next to e in the file.

Definition 4 (Entry locality under a pin)**.** A program is *entry-local under pin* π if, for every record q and every subset $U \subseteq R'$ that contains all entries reported for q in the full run, running it on $(\{q\}, U)$ with its normalizers set to $\pi(R')$ prints exactly q's lines from the full run.

Entry locality is what makes exact replay possible. It is a property of the program and its options, not something TARE proves; TARE tests it before and during every replay and refuses when a test fails. We assume the program is deterministic for fixed options and thread count, and that every input it reads is named on its command line.

4 Design

TARE updates a cached annotation in six stages (Figure 3). The first three decide *whether* exact replay is possible and *how*; the last three perform it and check it. Any failed decision or check leads to a full stock run.

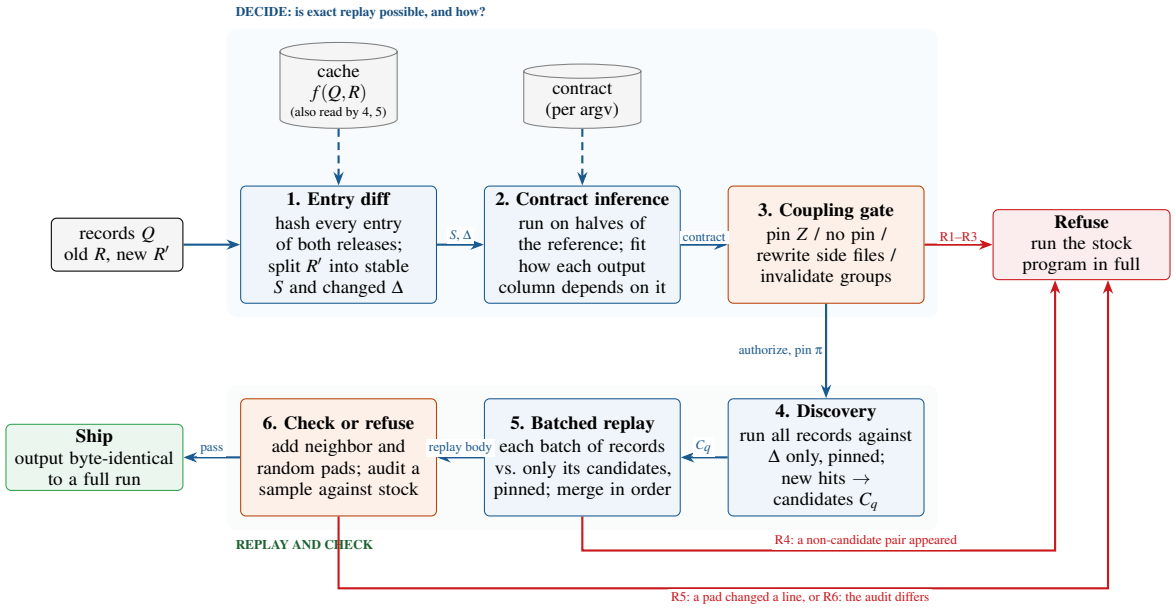


Figure 3: **How TARE updates an annotation.** Blue boxes do work; orange boxes are decision points. The top row decides whether the program's coupling can be neutralized; the bottom row reruns the unmodified program on candidates only and checks the result. Dashed arrows read stored state. Every red arrow (rules R1–R6) leads to a full stock run, so a user always receives the program's own output.

4.1 Stage 1: Entry Diff

TARE splits both releases into entries and computes a strict hash of each entry's scoring content, ignoring metadata such as dates and comments. Entries of the new release whose hash appears in the old one are stable; all others are changed. A cache keyed on the whole database file would miss on every entry after a release; a cache keyed on entry hashes keeps every stable entry, 86.6% of Pfam for 38.1→38.2.

4.2 Stage 2: Contract Inference

TARE must learn how the program's output depends on the reference without reading the program's code. It runs the program on a sample of records three times: against the whole reference, and against each half of it. It then lines up the output lines of the three runs by record and entry name.

Does the output split cleanly? If every line of the whole run appears in exactly one half-run, the output *decomposes* over entries: each line comes from one entry. If the whole run reports a line that neither half reports, entries interact in ways TARE cannot replay, and it refuses. The opposite case, a half reporting a line the whole omits, happens when entries compete and suppress each other; TARE accepts it only when the user declares which entries compete (Section 4.3).

How does each number depend on the reference? For each numeric column, TARE tests a small fixed set of hypotheses: the column does not depend on the reference at all (*identity*), or it scales with the number of entries, their total length, or the number of hits per record. A hypothesis fits if rescaling every half-run value by the corresponding ratio reproduces the whole-run value within the printed precision. A column that fits no hypothesis causes refusal. Columns that are ranks or labels rather than measurements are recognized and left to the program to reprint.

The result, called a *contract*, records how the output splits and how each column depends on the reference. It is computed once per command and reused for every later release.

4.3 Stage 3: The Coupling Gate

The gate turns the contract into a decision. Users may add declarations, which are data, never program names.

Size dependence → pin. If a column scales with the reference size, TARE needs a program option that sets that size. Users can declare it (`--pin '-Z {entry_count}'`), or TARE can find it by reading the program's help text and trying each numeric option until one moves the column by exactly the predicted ratio. With the option set to the new release's value, the program prints on a subset the numbers it would print on the whole.

No size dependence → no pin. If every column fits identity, TARE replays without a pin. This is the case for Infernal under Rfam's recommended command, which derives its search size from the query sequence instead.

Side files and competing entries. Rfam's recommended command passes a "clan" file listing families that compete.

On a subset, TARE rewrites that file to the families present, because the program rejects names it cannot find. If any member of a declared group changes, TARE re-searches the whole group, because a change to one member can reveal a hit of another that the cache never recorded.

The gate applies three rules in order and stops at the first that fires:

$$D = \begin{cases} \text{REFUSE} & \text{no contract} & \text{(R1)} \\ \text{REFUSE_PIN} & \text{size-dependent, no pin} & \text{(R2)} \\ \text{REFUSE_PIN} & \text{in-situ check fails} & \text{(R3)} \\ \text{AUTHORIZE} & \text{otherwise} & \end{cases} \quad (1)$$

R1 fires when the output does not split or a column fits no hypothesis. R2 fires when a column depends on the database size and no pin is available; TARE never guesses. R3 replays the probe sample against its own hits with the pin set and requires every byte to match the whole run.

4.4 Stage 4: Discovery

Changed entries may produce hits the cache has never seen. TARE runs the program on all records against the changed entries Δ alone, with the pin set. A record's *candidates* are its cached hits on stable entries plus its new hits:

$$C_q = \{e \in S : (q,e) \in f(Q,R)\} \cup \{e : (q,e) \in f_\pi(Q,\Delta)\}. \quad (2)$$

Discovery compares every record with every changed entry, so its cost is proportional to the changed fraction of the reference. It is the one cost TARE cannot avoid (Section 4.7).

4.5 Stage 5: Batched Exact Replay

Cached lines cannot be returned as they are, because their numbers were computed for the old release. TARE instead reruns the program on the candidates with the pin set, and the program prints the new numbers itself. Rerunning every protein against all candidates of all proteins would waste work: on one genome, the proteins' candidates together cover 3,719 families, but a typical protein has only two. TARE therefore groups proteins into batches, runs each batch against only its own proteins' candidates, and stitches the outputs back together in the original order. It chooses the grouping that minimizes

$$T_{\text{replay}} = \alpha k + \beta \sum_b |U_b| + \gamma \sum_b |B_b|\,|U_b|, \quad (3)$$

where k is the number of batches, $|B_b|$ and $|U_b|$ the proteins and families in batch b, α the cost of starting the program, β the cost per family of preparing a sub-database, and γ the cost of one protein–family comparison. Stitching is safe for HMMER because, in its source code, each protein's output block depends only on that protein's own hits, including the column widths and the per-protein statistics. As a tripwire, if a replay ever reports a protein–family pair outside the candidates, TARE refuses (R4), because a full run could contain pairs the cache never saw.

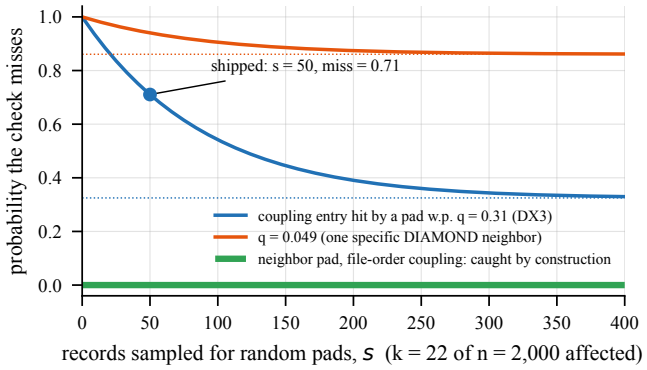


Figure 4: **How strong is the check?** Probability that the random pads miss an unknown dependency, from Equation (4) with the measured DIAMOND case ($k = 22$ affected of $n = 2{,}000$). The neighbor pad (green) catches file-order coupling with certainty; the random pads are a weaker, statistical safety net.

4.6 Stage 6: Check or Refuse

The replay assumes the program is entry-local. The check tries to prove that assumption wrong on the actual run. It reruns the replay with *pads*, families that were deliberately left out, added back to the subset. If the program is entry-local, pads change nothing: no candidate line changes and no pad family is reported. If anything changes, the replay depended on what it left out, and TARE refuses (R5).

TARE uses two kinds of pads. The *neighbor pad* adds, for every replayed protein, the families that sit next to its candidates in the database file; it catches neighbor coupling by construction, the failure we found in DIAMOND (Section 6.4). *Random pads* add three random samples of left-out families, each as large as the candidate set, for a fixed sample of $s = 50$ proteins. They test for dependencies of unknown form. If an unknown dependency affects k of n proteins and a random pad includes the responsible family with probability q, the check misses it with probability

$$P_{\text{miss}} = h + (1-h)(1-q)^3, \qquad h = \binom{n-k}{s} \Big/ \binom{n}{s}, \quad (4)$$

where h is the chance that the protein sample contains no affected protein (Figure 4). The check is a test, not a proof. As a final safeguard, an audit reruns the stock program on a random sample of proteins and refuses if any line differs (R6).

4.7 Cost Model

Let T_{stock} be the time of a full run on the new release and $f_\Delta = |\Delta|/|R'|$ the fraction of entries that changed. TARE's update time is approximately

$$T_{TARE} \approx a f_\Delta T_{\text{stock}} + c\,T_{\text{stock}}, \tag{5}$$

where the first term is discovery ($a \approx 1$ if the program's time grows linearly with database size) and c collects every other cost (replay, check, audit, parsing) as a fraction of a stock run. The speedup $1/(af_\Delta + c)$ can never exceed $1/f_\Delta$, and even for a tiny release it cannot exceed $1/c$. The first run also fills the cache, which costs a full run plus the contract probes; with fill time F and update time U, TARE becomes cheaper than repeated full runs after $n \geq (F - T_{\text{stock}})/(T_{\text{stock}} - U)$ releases.

5 Implementation

TARE is about 12,000 lines of Python with over 300 tests. Its core contains no program names: small parsers read profile databases, covariance-model databases, FASTA files, and tabular outputs, and users declare pins, side files, groups, and reporting thresholds as data. The cache is a SQLite file keyed by entry hash and record. A typical invocation wraps the normal command:

```
tare run --strategy reference --pin '-Z {entry_count}' \
  --cache cache.sqlite -- hmmscan --cut_ga --noali --cpu 4 \
  --tblout {output:tblout} --domtblout {output:domtblout} \
  {reference} {input}
```

6 Evaluation

We ask four questions. **Q1:** Is TARE's output identical to a full run? **Q2:** How much faster is an update, and does the cost model predict it? **Q3:** Does TARE refuse when replay would be wrong? **Q4:** How does it compare with existing tools?

Method. Every experiment followed the same protocol, designed so that a failure means the method failed rather than that the test was blind. We first read the program's source for behaviors that could make a comparison fail for unrelated reasons, then ran a pilot on separate data, then committed the design and pass/fail rules to version control before the measured run. Each test included a case that must pass and a case that must fail, and a minimum number of output lines below which a result counts as inconclusive. Measured runs used genomes drawn by a fixed random seed and never used before. One rule changed after its pre-registration, during a measured run; we disclosed the change and reran every control (Section 6.4).

Setup. HMMER 3.4 with Pfam 38.1 and 38.2; Infernal 1.1.5 with Rfam 15.0 and 15.1; DIAMOND 2.2.5 and BLAST+ 2.14.1 with Swiss-Prot 2026_01 and 2026_03; queries are bacterial genomes from NCBI RefSeq. Server timings use a dedicated AMD EPYC 7542 node; workstation timings use an Apple laptop and are indicative.

6.1 Metrics and Statistical Analysis

We define every reported quantity before using it. Throughout, "mean" is the arithmetic mean, $\bar{x} = \frac{1}{n}\sum_i x_i$; "median" is the sample median; the sample standard deviation s uses $n-1$ in its denominator; and "geometric mean" is $\left(\prod_{i=1}^{G} x_i\right)^{1/G}$, which we use only to summarize ratios.

M1. Exactness. For one run, let L be the stock run's body lines and L' TARE's. The run is EXACT if $L' = L$ as byte sequences; otherwise we count the differing positions $D = |\{i : L'_i \neq L_i\}|$ (plus any length difference) and classify each difference as whitespace-only, value-only, missing, extra, or reordered. Across experiments we report N, the total number of lines compared. When $D = 0$ over N lines, the one-sided 95% Clopper–Pearson upper bound on the per-line error rate is

$$p_{\text{up}} = 1 - 0.05^{1/N}. \tag{6}$$

This bound treats lines as independent, which they are not (lines of one run share a pin and a reference), so it is descriptive, not a guarantee.

M2. Pin necessity. For the unpinned control, the fraction of lines that differ from stock is $d = D/N$, reported with a 95% Wilson score interval.

M3. Wall time. T is the elapsed wall-clock time of one complete invocation, measured with a monotonic clock around the process, and includes start-up, parsing, and all program calls. Database preparation (`hmmpress`) before either arm is excluded from both.

M4. Speedup. For genome g with r runs per arm, alternating stock and TARE,

$$S_g = \frac{\text{median}_{i \leq r}\, T_{g,i}^{\text{stock}}}{\text{median}_{i \leq r}\, T_{g,i}^{TARE}}, \tag{7}$$

a ratio of medians, which is robust to a single slow run. On the workstation $r = 1$. Across G genomes we report the median of S_g, the geometric mean, and the range. For the server headline ($G = 10$, $r = 3$), the 95% confidence interval of the median is a percentile bootstrap over genomes (10,000 resamples, fixed seed). Run-to-run variability per arm is the coefficient of variation $\text{CV} = s/\bar{T}$.

M5. Cumulative speedup (record side). Over a collection processed in sequence, the cumulative speedup is a ratio of sums, $\sum_g T_g^{\text{stock}} \big/ \left(\sum_g T_g^{TARE} + P\right)$, where P is the one-time cost of contract probes; we also report it without P.

M6. Overhead of other tools. For a baseline tool, each paired run j gives $\rho_j = T_j^{\text{tool}}/T_j^{\text{stock}}$; we report $\bar{\rho} \pm s$ and the 95% t-interval $\bar{\rho} \pm t_{0.975,n-1}\, s/\sqrt{n}$.

M7. Cost-model error. For a predicted time $\hat{T}$ and a measured time T, the relative error is $\varepsilon = |\hat{T} - T|/T$; we report the maximum over genomes.

M8. Agreement of approximate output. For a tool that is not exact, we report the fraction of shared lines that are byte-identical (with a Wilson interval), and the agreement of

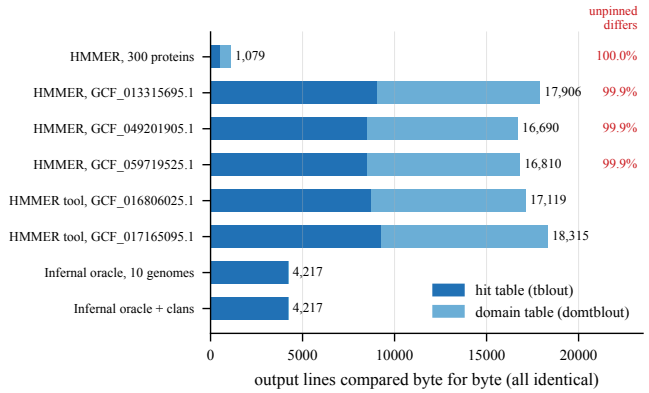


Figure 5: **Every line identical.** Output lines TARE produced in each exactness experiment, all byte-identical to a stock run on the new release. Red: in the same experiments, replaying without the pin changed nearly every line.

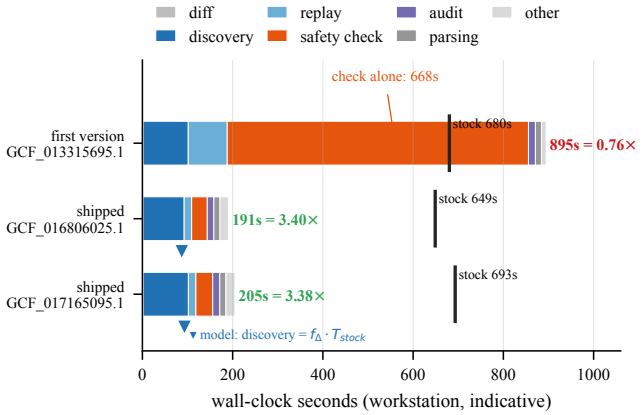


Figure 6: **Where the time goes** (workstation). Each bar is one update, split into stages; the black tick is a stock run on the same genome. Top row: medians of three alternating runs per arm; bottom rows: single runs. The first version (top) spent 668 s in its safety check and was slower than stock. Batching and sampling (bottom two, fresh genomes) cut the update to under a third of a stock run. Triangles: the cost model's prediction for discovery.

its E-values with stock over shared lines using Pearson's r and Kendall's tie-corrected rank correlation τ_b.

M9. Safety outcomes. Each safety run is EXACT (replay equals stock and every check passed), SAFE-REFUSE (a check failed and TARE ran the program in full; *true* if the replay differed from stock, *needless* if it did not), or UNSAFE (replay differed and every check passed). Any UNSAFE result was pre-registered as a failure of the safety claim. The probability that the random pads miss a dependency is Equation (4).

6.2 Q1: Is the Output Exact?

Figure 5 shows every exactness experiment. In total, TARE's HMMER outputs comprised 44,666 hit-table lines and 43,253 domain-table lines, and its Infernal outputs 4,676 lines; not one byte differed from the stock run ($D = 0$ over $N = 92{,}595$ lines). By Equation (6), a per-line error rate above 3.2×10^{-5} would be unlikely to produce this result (one-sided 95%). The experiments grew in scale: 300 proteins (534 hit lines), then three whole genomes (26,127 hit lines), then two more whole genomes through the shipped tool with batching and the check switched on. In every HMMER experiment, the same replay without the pin changed nearly every line: pooled over the three whole genomes, 26,094 of 26,127 hit lines differed ($d = 99.87\%$, Wilson 95% CI 99.82–99.91%), and on 300 proteins all 534 did. The pin is therefore necessary and, with `--cut_ga`, sufficient. The server headline adds [PENDING: ten genomes].

Where exactness ends. When HMMER reports hits by E-value threshold instead of `--cut_ga`, the replay matched on 300 proteins but failed on 1,000, differing on 3 hit lines and 5 domain lines. Hits near the threshold move in or out of a protein's list, which changes that protein's statistics; this mode is not entry-local and we do not claim it. The early pass on 300 proteins is why we require a minimum number of lines before accepting any result.

A second program. Infernal, which searches RNA families, behaved as its source code predicted: under Rfam's recommended command every numeric column fits identity, so no pin is needed. Replaying each genome's hits was byte-identical on ten genomes (4,217 lines) with and without clan competition. In that command, overlapping hits of competing families are marked rather than deleted, so removing a family cannot reveal a hidden hit; removing the family `GlmZ_SraJ` from one genome's reference was exact through the shipped tool (459 lines). With the non-default option `--oskip`, which deletes marked overlaps, the same removal was refused, because 14 overlap marks depended on families that had no remaining line. The shipped result on ten fresh genomes is [PENDING: Infernal measured ten].

6.3 Q2: How Much Faster?

Figure 6 shows that the replay and check designs decide whether TARE is faster than a full run. Our first version replayed every protein against all candidates and checked the result with four more full-size replays. On a workstation, with three alternating runs per arm, it took a median of 895 s against a median stock time of 680 s ($S_g = 0.76\times$; CV 3.7% for TARE and 1.7% for stock); the check alone took a median of 668 s. Batching made the replay 5.2× cheaper and sampling made the check 17.9–19.3× cheaper (ratios of the first version's median check time to each shipped run's). On two fresh genomes of 4,583 and 5,019 proteins, TARE replayed in 107 and 111 batches and took 191 s and 205 s against stock

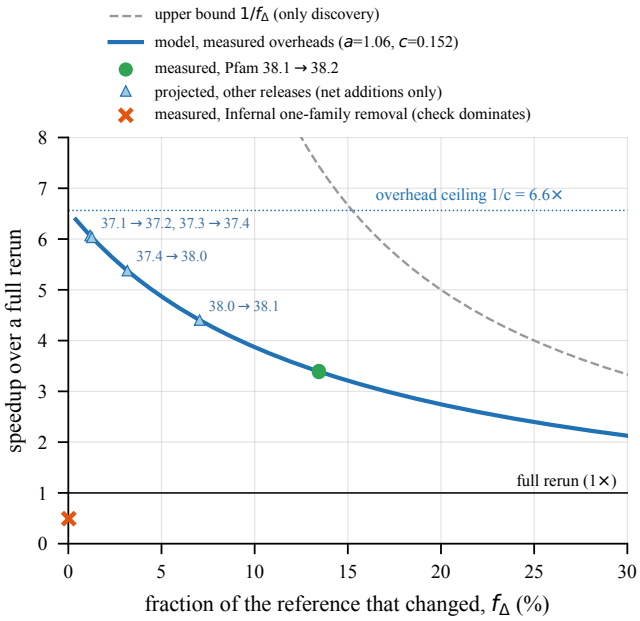


Figure 7: **The cost model across release sizes.** Speedup $1/(af_\Delta + c)$ with a and c the arithmetic means of the values measured on two genomes (blue), its upper bounds (dashed and dotted), the measured Pfam points (green), projections for other Pfam releases from their published family counts (triangles; edits not published, so these are optimistic), and Infernal, where the check dominates (cross).

runs of 649 s and 693 s: $S_g = 3.40\times$ and $3.38\times$ (Equation (7) with $r = 1$; geometric mean $3.39\times$), with byte-identical output. These are single runs on a shared machine, so we report no confidence interval for them; the server headline repeats each arm three times.

The cost model. With $f_\Delta = 0.134$, the discovery term predicts 87 s and 93 s on those genomes; discovery measured 91 s and 100 s, a relative error (M7) of 4.3% and 7.1%. The remaining costs, as a fraction of each stock run, average $c = 0.152$ (arithmetic mean of 0.153 and 0.151), so even a release that changed nothing could be at most $6.6\times$ faster with today's overheads (Figure 7); the check, audit, and parsing are therefore the next targets. Pfam 38.1→38.2 added more families than any release in Figure 1, so it is a conservative case: smaller releases project to 4.4–$6.1\times$. The cache fill took 871 s and 852 s (single runs), 1.2–$1.3\times$ a stock run, so TARE pays for itself from the first update (projected from these indicative times).

Server headline. On a dedicated node, across [PENDING: ten] fresh genomes with three alternating stock and TARE runs each, the median of S_g is [PENDING: X× (95% bootstrap CI; geometric mean; range)], the largest per-arm CV is [PENDING:], and the maximum relative error of the pre-registered prediction is [PENDING:].

When the check dominates. The speedup does not yet carry over to Infernal. Only 457 of Rfam 15.1's 4,227 families (10.8%) changed, yet in the measured single-family removal the update took 666 s against a 328 s stock run: exact, but $0.49\times$; the cache fill took $5.2\times$ a stock run. A bacterial genome is one DNA sequence or a handful, so the audit's sample and the random pads cover the entire genome; in the pilot, the audit alone took as long as a stock run. The check is sized by number of records, but its cost is set by sequence length. For inputs with few, long records it must be sized by length instead, which we have not yet done.

Table 2: Safety controls under the shipped check. Every case known to be wrong was refused, the correct case passed, and no run produced wrong output.

Control	Expected	Result
DIAMOND known-bad (6 lines missing)	refuse	REFUSE
DIAMOND regenerated (22 lines differ)	refuse	REFUSE
Injected 5% candidate drop	refuse	REFUSE
HMMER whole genome (correct)	pass	pass, exact
Fresh DIAMOND sample	no wrong output	SAFE-REFUSE
BLAST+ boundary	refuse	REFUSE (R1)

6.4 Q3: Does TARE Refuse When It Should?

A failure, and what it revealed. The check's design came from a failure. When we declared DIAMOND's database-size option (`--dbsize`) as a pin, the replay on 300 proteins lost 6 of 31,377 stock lines while every other line was byte-identical, and our first check, which used only random pads, passed it. Under our pre-registered rule, that was an UNSAFE result and invalidated that check. To find the cause, we added left-out entries back in halves until each missing line returned. Every one of the six lines was restored by exactly one entry: the candidate's neighbor in the database file. Figure 8 shows the mechanism in DIAMOND's source: its first filter reads bytes across the boundary into the preceding entry. DIAMOND's results therefore depend on the order of the database, and no setting can make a subset replay exact. A random pad contains one particular neighbor with probability 0.049, so three random pads miss it 86% of the time, which is why the first check passed.

The shipped check. Adding the neighbor pad closed this gap (Table 2). On a fresh sample of 2,000 proteins, the replay differed from stock in 22 of 227,223 lines (20 missing, 2 extra), (0.0097% of lines), and the check failed on all five pads: a true SAFE-REFUSE. A second fresh DIAMOND sample was also refused. BLAST+ was refused earlier, at contract inference: one of 15,125 aligned lines fit no hypothesis, so R1 fired. The correct HMMER case passed with no needless

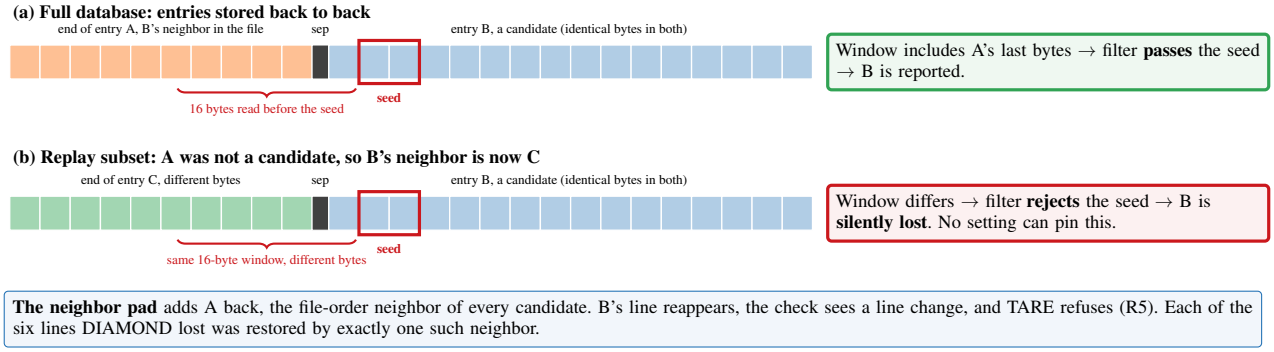


Figure 8: **Why DIAMOND cannot be replayed exactly.** DIAMOND stores database entries back to back, separated by a single byte. Its first filter examines a window of 16 bytes before and 32 after each short matching "seed" without stopping at the entry boundary (DIAMOND 2.2.5, `finger_print.h`). A seed near the start of entry B therefore reads the end of whichever entry precedes B. Removing that neighbor from the database changes the window and can drop B's hit. The neighbor pad detects exactly this dependency.

refusal.

Limits. For dependencies other than file-order neighbors, the random pads are weak: with 22 affected proteins of 2,000, the shipped sample of 50 misses with probability 0.71 (Figure 4). Our safety claim therefore rests on the neighbor pad, the in-situ check, and the audit, together with zero wrong outputs across all controls.

A disclosed change. During the measured run of the sampled check, we changed one rule so that a candidate of one protein that is also a neighbor of another counts as a candidate rather than a pad hit, after it caused a needless refusal. Because that change made the check more permissive, we disclosed it and reran every control under the final code. Every known-bad case was still refused, for the same reason as before.

What inference cannot do. On HMMER, TARE found the `-Z` option on its own from held-out data, but could not infer reporting thresholds, so users declare them. On DIAMOND, it found no size option. Neither led to wrong output, because a missing pin is a refusal (R2).

6.5 Q4: Comparison with Existing Tools

General-purpose caches. Riker, INCR, and ProcessCache all re-executed HMMER on a new genome, as their designs require, because the input bytes changed (Table 3). Riker skipped an identical rerun in the same container; INCR re-executed even an identical rerun, because it records the command's output file as a dependency that does not yet exist and rejects it once the file exists. Tracing also costs time on the first run (M6, $n = 4$ paired runs each): Riker took 1.62× the stock time (95% CI 1.53–1.71) and INCR 2.24× (95% CI 2.06–2.43); both intervals exclude 1. These systems are correct for what they are designed to do; a database release is simply outside it.

Table 3: General-purpose caches on HMMER. "New input": behavior when the query or database changes. First-run overhead: arithmetic mean of $n = 4$ paired time ratios against stock, with 95% t-interval, in a uniform container (500 proteins, all 30,134 families); TARE's is its cache fill on the workstation (two single runs).

System	New input	First-run overhead
Riker [8]	reruns	1.62× [1.53, 1.71]
INCR [27]	reruns	2.24× [2.06, 2.43]
ProcessCache [24]	reruns	—
TARE	replays changed part	1.2–1.3×

Approximate incremental search. Specialized incremental search does reuse work across a release, but not exactly. We ran iSeqSearch's incremental DIAMOND [28] on the same 300 proteins as our DIAMOND experiment, updating Swiss-Prot from 2026_01 to 2026_03, and compared it with a full run (Figure 9). Its E-values are close (over 31,377 shared lines, Pearson $r = 0.999999$ and Kendall $\tau_b = 0.99999$), but only 407 of those lines (1.30%, Wilson 95% CI 1.18–1.43%) are byte-identical; 30,968 report the same alignment with different printed numbers, 2 report a different alignment, and 133 lines are hits to sequences the new release removed. TARE refuses this program and returns the full run; where it replays, every line is identical.

Record-side reuse. The same machinery applies when the *proteins*, not the database, are new. On SnpEff variant annotation [7], reusing earlier records gave a speedup of 1.17× on a dedicated node (median of 10 alternating paired ratios; 95% percentile bootstrap CI 1.159–1.171 from 10,000 resamples, which excludes 1), with byte-identical output on 52,638 records; SnpEff's start-up time limits that case to about 1.3×. On HMMER over collections of 30 and 40 *E. coli* genomes,

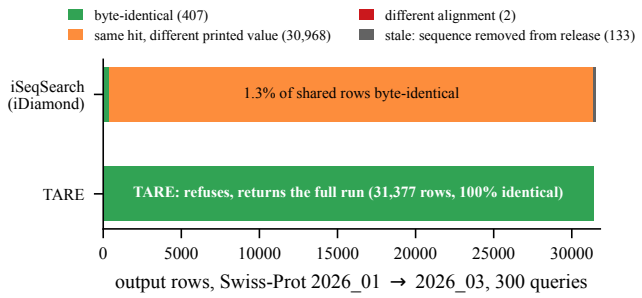


Figure 9: **Approximate versus exact.** Output of iSeqSearch's incremental DIAMOND after a Swiss-Prot update, compared line by line with a full run. Its numbers are close, but only 1.3% of lines match exactly and 133 lines report sequences the new release deleted. TARE refuses this program and returns the full run.

reusing proteins searched earlier gave cumulative speedups (M5, ratio of summed times) of 2.04× and 4.94×, or 1.66× and 3.53× including the one-time probe cost $P = 1{,}595$ s.

7 Discussion

Threats to validity. *Generality:* the two programs TARE replays exactly, HMMER and Infernal, come from one research group and share code, so they may share the property that makes replay work; the two refused programs come from other groups. We therefore claim that the method works for programs whose coupling lies in settings they expose, and that it detects when coupling lies elsewhere; we do not claim it works for most search programs. *Release choice:* we measure one release pair per database; smaller releases should be faster, as the model projects, but we have not measured them. *Timing:* workstation numbers are single runs on a shared machine; the headline is measured on a dedicated node with alternating runs. *Check strength:* the random pads are weak (Figure 4).

Anticipated objections. *"Why not just rescale the cached E-values?"* Rescaling reproduces the program's arithmetic but not its rounding, and it must be written for each program and column; pinning lets the program print its own numbers. *"InterProScan already reuses precomputed results."* Its lookup is keyed by the protein sequence for a fixed database version; it answers "has this protein been seen?" rather than "what changed in the database?" *"Use a faster or GPU search program."* That is complementary: it shrinks T_{stock}, and TARE's savings apply on top whenever the program's coupling can be pinned.

Deployment. The natural deployment is InterProScan [12] after a member-database update. Member databases that score each model against its own threshold, as Pfam does, should behave like our HMMER results; those that compare hits across models would need declared groups or would be refused.

8 Related Work

General-purpose incremental execution. Build systems and workflow managers avoid repeated work by keying a command on the bytes it reads (Table 4). Riker traces system calls to rerun only commands whose inputs changed [8]; ProcessCache caches process executions [24]; INCR memoizes shell command effects and can split inputs into annotated chunks [27]; Nextflow, Snakemake, and Cromwell hash task inputs [3, 10, 17]; and Mokhov et al. model the design space [16]. Their unit of reuse is the command, so they cannot reuse part of an output whose numbers depend on the whole database. TARE is complementary: it decides what to rerun inside one command.

Incremental sequence search. iBLAST searches only appended sequences and corrects BLAST's E-values with a formula derived for BLAST [9]. iSeqSearch applies that correction to other programs with BLAST-style output, including DIAMOND [28]. GeStore runs unmodified BLAST and HMMER through administrator-written plugins that rewrite inputs and merge outputs [20]. MMseqs2 supports incremental clustering [25], and InterProScan and eggNOG-mapper reuse precomputed matches keyed by protein sequence [4, 12]. These approaches are hand-built for one program, approximate when the database changes, or keyed by query rather than database. Our DIAMOND result applies directly to them: no correction of DIAMOND's printed numbers can recover hits its filter dropped because a neighboring entry was absent.

Black-box decomposition and incremental computation. KumQuat synthesizes combiners that let an unmodified command run on pieces of its input stream [23], and Caruca classifies opaque commands by whether they keep state [14]. Incremental view maintenance and self-adjusting computation keep results current under input changes but require the computation to be written in their framework [1, 2]. These methods split an input stream or a known program; TARE splits a database argument whose size enters every printed number, for a binary it cannot inspect. Its partition probes borrow the spirit of metamorphic testing [6, 22] rather than claiming a new testing method.

9 Conclusion

Reference databases change every few weeks to months, and the programs that search them start from scratch every time. We showed that for an important class of programs this is unnecessary: the coupling between database entries lies in a few numbers the program already lets users set, and pinning those numbers lets an unmodified program reprint, byte for byte, the output of a full run while searching only what changed. TARE discovers that coupling, replays exactly where it can, and refuses where it cannot, as it did for DIAMOND, whose filter reads neighboring entries. Every output TARE produced across HMMER and Infernal was identical to a full rerun, and

Table 4: Prior work on the problem of updating annotations after a database release. "Unmodified": runs the program as is. "Release-aware": reuses work when the database changes. "Identical": output equals a full run byte for byte. "Per-tool code": requires code or formulas written for each program.

System	Grain	Unmodified	Release-aware	Identical	Per-tool code	Limitation for this problem
Riker, ProcessCache, INCR [8, 24, 27]	command / chunk	yes	no	yes	no	a release reruns everything
Nextflow, Snakemake [10, 17]	task	yes	no	yes	no	task inputs change
iBLAST [9]	appended sequences	yes	growth only	rescaled	yes	BLAST-specific E-value formula
iSeqSearch [28]	appended sequences	yes	growth only	rescaled	yes	1.3% of lines identical; stale hits
GeStore [20]	plugin	yes	yes	rescaled	yes	administrator-written plugins
Precomputed lookups [4, 12]	query sequence	yes	no	yes	no	keyed by query, fixed release
KumQuat [23]	input stream	yes	no	yes	no	no database argument
IVM, self-adjusting [1, 2]	operator	no	yes	yes	n/a	computation must be rewritten
TARE	record × entry	yes	yes	yes	no	refuses programs with hidden coupling

a Pfam update ran [PENDING: X×] faster than a full rerun on a dedicated server. Extending TARE to InterProScan's member databases, and sizing its check by sequence length rather than record count, are the next steps.

Availability

An anonymized artifact accompanies this submission. It contains TARE, every pre-registration and addendum, the result files from which each number in this paper is taken, and the scripts that regenerate every figure from those files. The repository URL is omitted for double-blind review.

References

[1] Umut A. Acar, Guy E. Blelloch, and Robert Harper. Selective memoization. In *Proc. ACM SIGPLAN–SIGACT Symposium on Principles of Programming Languages (POPL)*, 2003.

[2] José A. Blakeley, Per-Åke Larson, and Frank Wm. Tompa. Efficiently updating materialized views. In *Proc. ACM SIGMOD International Conference on Management of Data*, 1986.

[3] Broad Institute. Cromwell call caching. https://cromwell.readthedocs.io/en/stable/Configuring/, 2024. Call caching; documentation.

[4] Carlos P. Cantalapiedra, Ana Hernández-Plaza, Ivica Letunic, Peer Bork, and Jaime Huerta-Cepas. eggNOG-mapper v2: Functional annotation, orthology assignments, and domain prediction at the metagenomic scale. *Molecular Biology and Evolution*, 38(12):5825–5829, 2021.

[5] I-Min A. Chen, Ken Chu, Krishnaveni Palaniappan, et al. The IMG/M data management and analysis system v.6.0: new tools and advanced capabilities. *Nucleic Acids Research*, 49(D1):D751–D763, 2021.

[6] T. Y. Chen, S. C. Cheung, and S. M. Yiu. Metamorphic testing: A new approach for generating next test cases. Technical Report HKUST-CS98-01, Hong Kong University of Science and Technology, 1998. arXiv:2002.12543.

[7] Pablo Cingolani, Adrian Platts, Le Lily Wang, Melissa Coon, Tung Nguyen, Luan Wang, Susan J. Land, Xiangyi Lu, and Douglas M. Ruden. A program for annotating and predicting the effects of single nucleotide polymorphisms, SnpEff: SNPs in the genome of *Drosophila melanogaster* strain w1118; iso-2; iso-3. *Fly*, 6(2):80–92, 2012.

[8] Charlie Curtsinger and Daniel W. Barowy. Riker: Always-correct and fast incremental builds from simple specifications. In *Proc. USENIX Annual Technical Conference (ATC)*, pages 885–898, 2022.

[9] Sajal Dash, Sarthok Rasique Rahman, Heather M. Hines, and Wu-chun Feng. iBLAST: Incremental BLAST of new sequences via automated e-value correction. *PLoS ONE*, 16(4):e0249410, 2021.

[10] Paolo Di Tommaso, Maria Chatzou, Evan W. Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35:316–319, 2017.

[11] Sean R. Eddy. Accelerated profile HMM searches. *PLoS Computational Biology*, 7(10):e1002195, 2011.

[12] Philip Jones, David Binns, Hsin-Yu Chang, et al. InterProScan 5: genome-scale protein function classification. *Bioinformatics*, 30(9):1236–1240, 2014.

[13] John Jumper, Richard Evans, Alexander Pritzel, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596:583–589, 2021.

[14] Evangelos Lamprou, Seong-Heon Jung, Mayank Keoliya, Lukas Lazarek, Konstantinos Kallas, Michael Greenberg, and Nikos Vasilakis. Caruca: Effective and efficient specification mining for opaque software components, 2025. arXiv:2510.14279.

[15] Wenjun Li, Kathleen R. O'Neill, Daniel H. Haft, et al. RefSeq: expanding the Prokaryotic Genome Annotation Pipeline reach with protein family model curation. *Nucleic Acids Research*, 49(D1):D1020–D1028, 2021.

[16] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build systems à la carte. *Proceedings of the ACM on Programming Languages*, 2(ICFP), 2018.

[17] Felix Mölder, Kim Philipp Jablonski, Brice Letcher, Michael B. Hall, Christopher H. Tomkins-Tinch, Vanessa Sochat, Jan Forster, Soohyun Lee, Sven O. Twardziok, Alexander Kanitz, Andreas Wilm, Manuel Holtgrewe, Sven Rahmann, Sven Nahnsen, and Johannes Köster. Sustainable data analysis with Snakemake. *F1000Research*, 10:33, 2021.

[18] Typhaine Paysan-Lafosse, Antonina Andreeva, Matthias Blum, Sara Rocio Chuguransky, Tiago Grego, Beatriz Lazaro Pinto, Gustavo A. Salazar, Maxwell L. Bileschi, Felipe Llinares-López, Laetitia Meng-Papaxanthos, Lucy J. Colwell, Nick V. Grishin, R. Dustin Schaeffer, Damiano Clementel, Silvio C. E. Tosatto, Erik Sonnhammer, Valerie Wood, and Alex Bateman. The Pfam protein families database: embracing AI/ML. *Nucleic Acids Research*, 53(D1):D523–D534, 2025.

[19] Typhaine Paysan-Lafosse, Matthias Blum, Sara Chuguransky, et al. InterPro in 2022. *Nucleic Acids Research*, 51(D1):D418–D427, 2023.

[20] Edvard Pedersen, Nils Peder Willassen, and Lars Ailo Bongo. Transparent incremental updates for genomics data analysis pipelines. In *Euro-Par 2013: Parallel Processing Workshops*, volume 8374 of *Lecture Notes in Computer Science*, pages 311–320. Springer, 2014.

[21] Pfam. Pfam release 38.2 release notes. https://ftp.ebi.ac.uk/pub/databases/Pfam/current_release/relnotes.txt, 2026. Accessed 2026-10-10.

[22] Sergio Segura, Gordon Fraser, Ana B. Sánchez, and Antonio Ruiz-Cortés. A survey on metamorphic testing. *IEEE Transactions on Software Engineering*, 42(9):805–824, 2016.

[23] Jiasi Shen, Martin Rinard, and Nikos Vasilakis. Automatic synthesis of parallel Unix commands and pipelines with KumQuat. In *Proc. PPoPP*, 2022.

[24] Kelly Renee Shiptoski. *Reproducibility and Performance Optimizations for Unmodified Linux Programs*. PhD thesis, University of Pennsylvania, 2023.

[25] Martin Steinegger and Johannes Söding. MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets. *Nature Biotechnology*, 35:1026–1028, 2017.

[26] The UniProt Consortium. UniProt: the universal protein knowledgebase in 2025. *Nucleic Acids Research*, 53(D1):D609–D617, 2025.

[27] Yizheng Xie, Evangelos Lamprou, Jerry Xia, and Nikos Vasilakis. Faster re-execution via bolt-on incrementalization. In *Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2026.

[28] Hyunwoo Yoo, Mohammad S. Refahi, Robi Polikar, Bahrad A. Sokhansanj, James R. Brown, and Gail L. Rosen. iSeqSearch: Incremental protein search for iBlast/iMMSeqs2/iDiamond. *PeerJ*, 2025.